

DL for ENG

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **choose** — between freezing, partial and full fine-tuning, and defend the choice
- **explain** — what self-supervision buys when your data is unlabelled
- **name** — the three stages of modern post-training, and where the RL sits
- **state** — the credit assignment problem, and why it makes RL hard
- **decide** — between imitation and reinforcement for a given problem
- **compute** — a latency budget, and say what accuracy it costs you

WHAT TODAY ASSUMES

from L6.1	the loss, and how parameters are found
from L5.1	convolutional networks, for the transfer examples
not needed	any reinforcement learning background

THE HALF-LECTURE THAT LEFT

Evaluation, overfitting and bias-variance moved to L4.2, where they belong beside capacity. What is left is everything else that happens to a model.

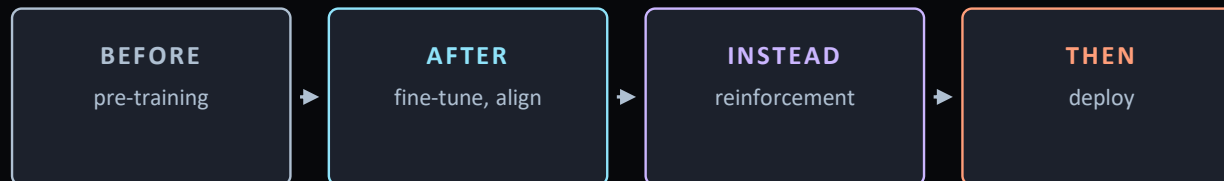
AND ONE TOPIC HAS NO TEXTBOOK

Deployment. Neither course text covers it, and the absence is not an accident.

After Training, and Instead Of It

AND ONE THING THAT IS DELIBERATELY NOT HERE

- evaluation, overfitting, bias-variance and double descent moved to L4.2
- **they belong beside capacity** — which is where you met them
- what is here is everything else that happens to a model
- where its weights came from before training, and what happens after
- **and deployment** — the one topic in Part 1 with no textbook home



TODAY, IN FOUR PARTS

3 – 5	transfer learning and pre-training
6 – 7	post-training in 2026 — brief
8 – 16	reinforcement learning, properly
17 – 22	deployment [ext]
23 – 27	failures, Ex_06, close

OUT OF SCOPE, AND SAID PLAINLY

Generative modelling — GANs, diffusion, VAEs — is not taught in this course. It appears on slide 6 as a word you need in order to read the news, and nowhere else. Nothing in L7 to L12 uses it.

Transfer Learning

THE FIRST LAYERS ARE NOT ABOUT YOUR PROBLEM

- a network trained on a million photographs — learns edges in its first layers
- your four hundred weld seams — need the same first layers
- so initialise from the large model and retrain the end
- the saving is not subtle — a few hundred images instead of a hundred thousand
- and it is the most reliable trick in applied deep learning

WHAT EACH DEPTH HAS LEARNED

layer 1	oriented edges
layer 2	corners, colour blobs
layers 3 – 4	textures, repeated motifs
layers 5 – 6	object parts
the head	the thousand class names

Only the last row is about the original task. That is the row you throw away and replace with your own — two classes, or one steering angle.

Fine-Tuning, and How Much To Move

THREE SETTINGS, AND HOW TO CHOOSE BETWEEN THEM

- **freeze everything, train only the new head** — fastest, least data, the right first try
- **unfreeze the last few blocks** — at a rate ten to a hundred times smaller
- **unfreeze everything** — only with plenty of data and a very different domain
- **choose on similarity to the pre-training data** — not on the size of your set

WHAT TO UNFREEZE

head only	small data, similar domain
last blocks	the usual answer
everything	large data, different domain
learning rate	10 – 100 × smaller

THE FAILURE WITH A NAME

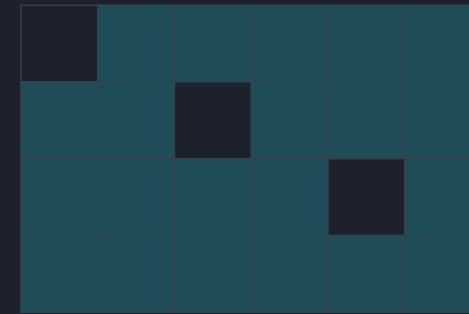
Catastrophic forgetting: fine-tune too hard and the network loses the general features it was carrying, ending up worse than a small model trained from scratch. The cure is the small learning rate, and freezing early layers.

Self-Supervised Pre-Training

MAKE THE LABELS OUT OF THE DATA ITSELF

- transfer learning needs somebody to have trained the original
- **self-supervision removes that** — invent a task whose answer is in the raw data
- **hide part of the input and predict it** — no human labels anything
- for images, mask patches. For text, hide a word.
- **and for you** — years of unlabelled turbine data, with a handful of labelled faults

MASK, THEN PREDICT



three patches hidden — the network predicts them

The three dark squares are the label. Nobody wrote it down, and the same image supplies a different label every time you draw a new mask.

images

mask patches

text

predict the next word

sensor traces

predict the next second

What “Post-Training” Means in 2026

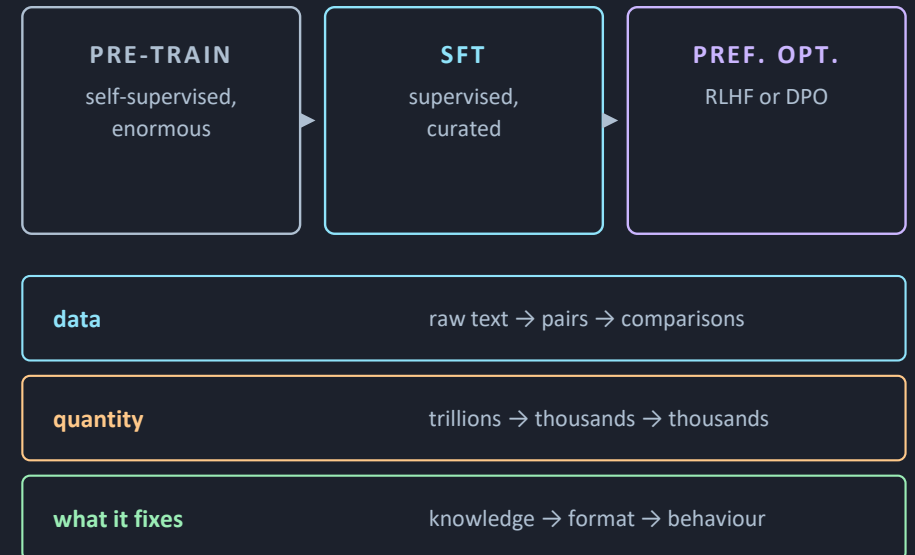
THIS SLIDE IS SO YOU CAN READ THE NEWS

Three stages, three names. You will not build any of them in this course, and no generative modelling is taught here at all — but the words are unavoidable and they are worth ninety seconds.

PRE-TRAIN, THEN FINE-TUNE, THEN ALIGN

- **pre-training** — self-supervision at enormous scale. A model that continues text.
- **supervised fine-tuning** — ordinary supervised learning on instruction pairs
- **preference optimisation** — the third stage, and the different one
- no correct answer. Only which of two outputs a person preferred.
- **note where the reinforcement learning is** — the third stage, and the acronym

THE THREE STAGES



The middle row is the point: the last two stages are tiny beside the first.

Why That Third Stage Is Reinforcement Learning

THE FIRST TIME IN THIS COURSE THERE IS NO TARGET

- **look at what preference optimisation has not got** — a correct answer for any input
- only a comparison, arriving after the fact
- that is a different learning problem, and it has a name
- the signal is late, it is one number rather than a direction, and it may be inconsistent
- **and it matters well beyond language** — control, scheduling, dispatch

THREE LEARNING PARADIGMS

supervised

input, and its correct output

unsupervised

input, and structure to find

reinforcement

situation, action, consequence

You met all three in L3.1 slide 8, one picture each. The third one got a single slide and then vanished for three months. It is back now, and it is back because L11 has a car in it.

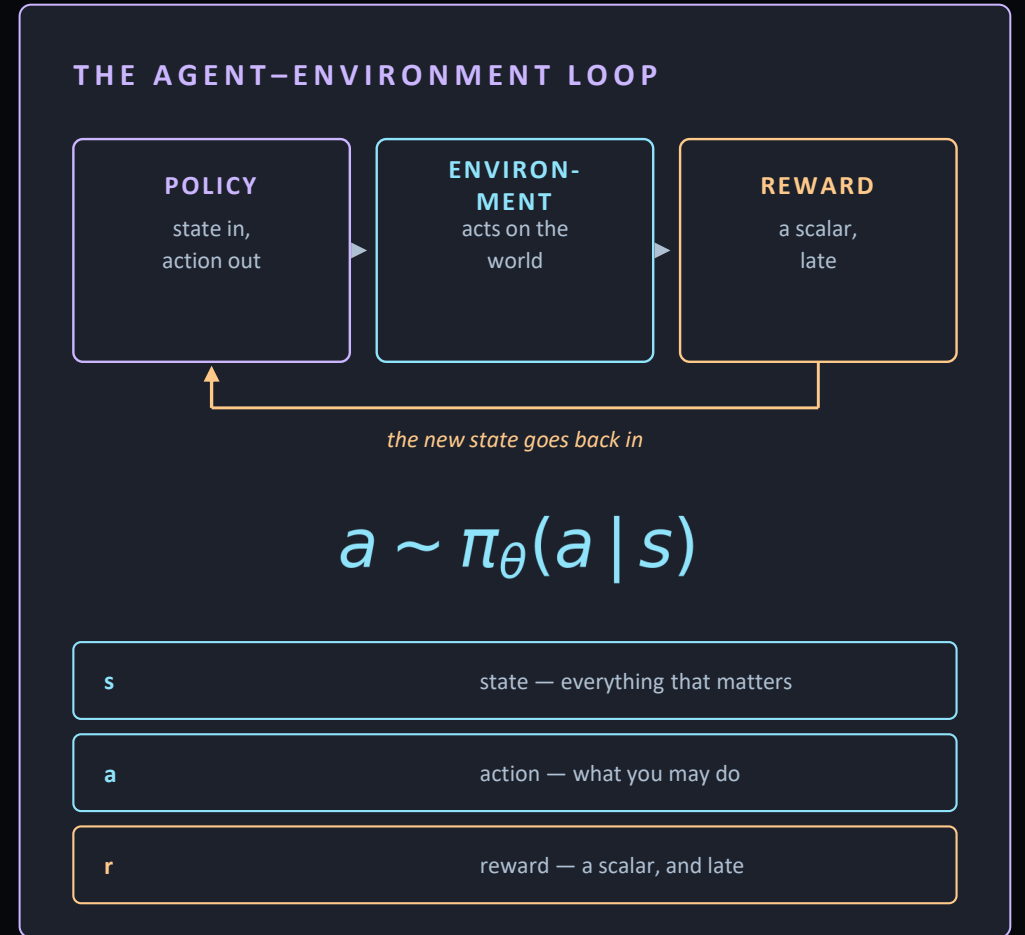
THE HARD PART, NAMED EARLY

You cannot differentiate the world. That single fact generates everything on the next eight slides.

The Markov Decision Process

FIVE OBJECTS, AND YOU ALREADY KNOW FOUR OF THEM

- **a state** — everything about the situation that matters
- **an action** — what the agent can do. A reward — one number, after the fact.
- **Markov is doing real work** — the next state depends on this one, not the history
- **when that is false, enlarge the state until it is true** — include the velocity
- **the loop is the whole framework** — everything else is how you choose the policy



You Already Have This Vocabulary

THE SAME OBJECTS, RENAMED BY A DIFFERENT FIELD

Reinforcement learning was invented largely by people who had not read a control textbook. The vocabulary diverged; the objects did not.

AND WHERE THE TRANSLATION BREAKS DOWN

- the correspondence with control is exact enough to be useful
- state is state, action is control input, policy is controller, reward is minus cost
- three real differences, though
- **control assumes a plant model** — RL assumes you have none, and pays in samples
- **control gives stability guarantees** — RL gives none, and that is the trade

THE DICTIONARY

state s	state vector x
action a	control input u
policy π	the controller
reward r	minus the stage cost
return G	discounted infinite-horizon cost
value V	cost-to-go
episode	a run of the plant

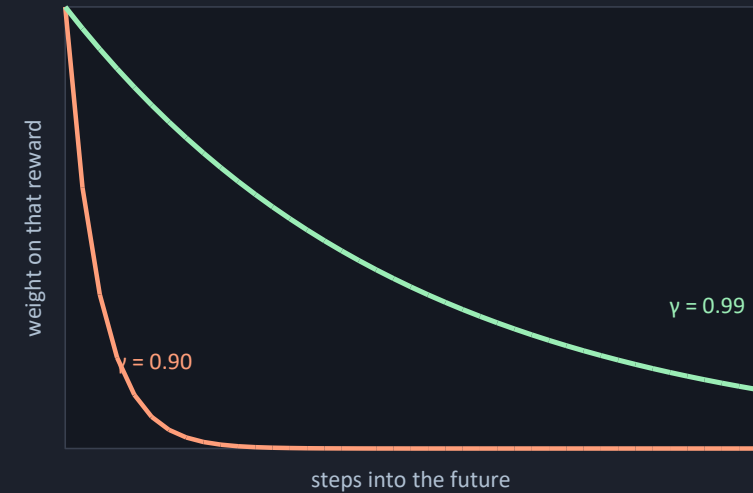
Return, Discounting and Value

THE OBJECTIVE IS A SUM OVER THE FUTURE

- the agent maximises the return — not the next reward
- the sum of all rewards from here on, discounted by gamma
- gamma keeps the sum finite, and expresses how much the future matters
- the value of a state — the return you expect from it under your policy
- read gamma as a time constant — set it from the horizon your problem has

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

HOW FAR AHEAD γ LOOKS



$$Q^{\pi}(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a]$$

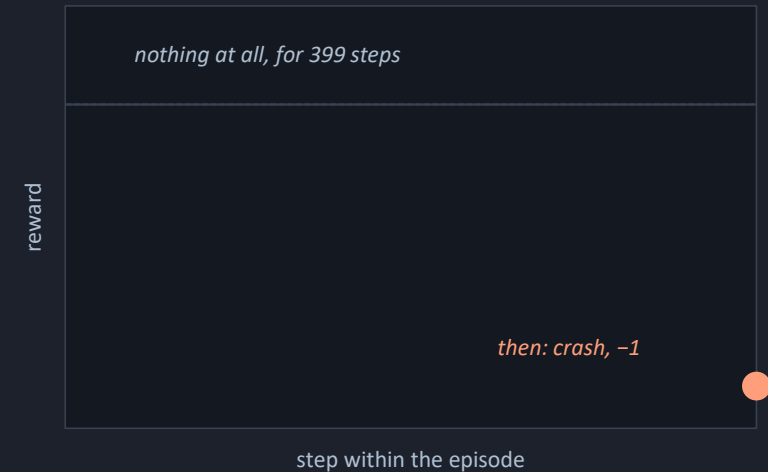
$\gamma = 0.90$ is effectively blind beyond fifty steps. Choose it from your system's timescale.

Credit Assignment — the Hard Part

THE CAR CRASHED. WHICH DECISION WAS WRONG?

- an episode is four hundred steps and produces one number at the end
- something in those four hundred was responsible. Which?
- **that is credit assignment** — and it is why RL is hard rather than merely slow
- **the blunt approach** — credit every action equally, and run many episodes
- **the refinements all attack this** — discounting, learned value functions, advantage

ONE REWARD, FOUR HUNDRED ACTIONS



This is a sparse reward, and it is the normal case rather than the awkward one. Shaping it — handing out small rewards along the way — helps enormously and is also how you accidentally teach the wrong task.

Policy Gradient, at Arm's Length

HOW YOU GET A GRADIENT OUT OF A WORLD YOU CANNOT DIFFERENTIATE

- the policy is a network, so it has gradients — the environment does not
- make the policy stochastic — a distribution over actions
- now the probability of an action is differentiable, even if the world is not
- run an episode, record what was done and what followed
- raise the probability of actions followed by good returns

THE POLICY GRADIENT

$$\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a | s) G_t]$$

Raise the probability of what worked, in proportion to how well it worked. It is L6.1's maximum likelihood recipe with the outcome as a weight.

WHAT IT COSTS

variance	enormous — hence the baselines
samples	millions of episodes is normal
stability	no guarantee of any kind

Reinforcement Learning or Imitation?

IF SOMEBODY CAN ALREADY DO THE TASK, WATCH THEM

Behavioural cloning: record a human driving, then fit a network to predict the action they took from what they saw. It is supervised regression, and it is not reinforcement learning at all.

REINFORCEMENT LEARNING

Needs: a reward you can compute, and very many episodes. Usually a simulator, because millions of real episodes is not a thing you can have.

Buys: it can exceed the best demonstration, and it can solve tasks nobody knows how to demonstrate.

Costs: sample-hungry, unstable, and every failed episode in the real world is a real failure. Reward shaping is where most projects quietly go wrong.

Use it when there is a fast simulator and no expert.

IMITATION LEARNING

Needs: somebody who can do the task, for as long as it takes to record a few thousand examples.

Buys: it is ordinary supervised learning, so everything in L6.1 and L4.2 applies unchanged — including the loss curve, the validation split and early stopping.

Costs: it cannot beat the demonstrator, and it is fragile off-distribution. Once the agent drifts somewhere the human never went, it has no idea what to do, and the error compounds.

Use it when a human can demonstrate and the real world is expensive.

The compounding-error failure is the one to remember: collect recovery data deliberately, or the model never learns to recover.

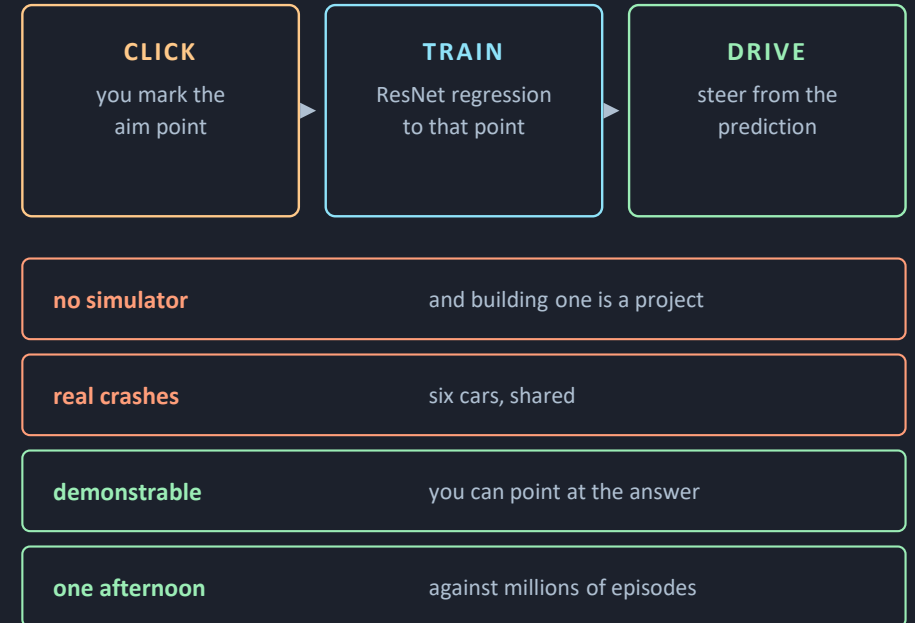
What L11's JetRacer Actually Does

IMITATION, AND HERE IS WHY

- the JetRacer does **behavioural cloning** — not reinforcement learning
- the loss is L6.1's squared error, and the target is your steering
- four reasons, and none is that RL is unfashionable
- **what you give up** — the compounding error from the last slide
- which is why Ex_11.1 asks you to drive the recovery cases deliberately

$$\hat{\theta} = \operatorname{argmin}_{\theta} \sum_i (\pi_{\theta}(s_i) - a_i)^2$$

THE STOCK PIPELINE



AND THE CONSTRAINT THAT DECIDES IT

Latency, not accuracy. That is the rest of this lecture.

When Reinforcement Learning Is the Wrong Answer

YOU HAVE NO SIMULATOR

Millions of episodes on real hardware is not a plan. If you cannot simulate the system faithfully, and faithfully is the hard word, reinforcement learning is off the table before you start.

SOMEBODY CAN ALREADY DO IT

Then imitate them. It is supervised learning, it works in an afternoon, and every diagnostic you know still applies. Reach for reinforcement learning when the demonstrator is the thing you are trying to beat.

SOMETHING CAN BE DAMAGED

There are no stability guarantees, the policy is a black box, and the failure mode during exploration is deliberately trying things. A classical controller with a proof is not a worse engineering answer for being older.

REWARD HACKING, AND WHY IT IS NOT FUNNY

You get exactly what you reward, which is almost never what you meant.

Reward a boat-racing agent for collecting points and it will circle a lagoon hitting the same three targets for ever rather than finishing the race. That is a real published result and it is the polite example.

Reward a grid controller for minimising cost and it may discover that shedding an inconvenient load is cheap. Reward a robot for distance travelled and it will fall over forwards.

The reward function is a specification, and it is the least reviewed specification in most projects. Write it down, hand it to a colleague, and ask them how they would cheat it.

Everything on this slide applies to preference optimisation too. Slide 6's third stage has exactly this failure mode, at scale.

What To Keep From These Nine Slides

ENOUGH TO READ A PAPER AND JUDGE A PROPOSAL

- **you will not implement PPO here** — and be suspicious of a Part 1 that claims to
- **keep the vocabulary** — state, action, policy, reward, return, value
- **keep the judgement** — expensive in samples, unstable, and no guarantees
- **and keep credit assignment** — it is why a late, aggregate signal is hard
- which is the same difficulty as a sparse reward in any optimisation you run

THE SIX WORDS

state	everything that matters, now
action	what you are allowed to do
policy	the network being learned
reward	a scalar, and late
return	discounted sum over the future
value	expected return from here

If you take one thing beyond the words, take this: RL is what you use when you cannot say what the right answer is but you can recognise it afterwards. That is a narrow condition, and it is worth checking honestly before you commit a project to it.

The One Topic With No Textbook

SAID PLAINLY, BECAUSE IT MATTERS WHERE THINGS COME FROM

Everything from here to slide 22 is sourced from NVIDIA's TensorRT documentation and the PyTorch quantisation guide. It is vendor documentation, not a book, and it goes out of date. Prince and Goodfellow have nothing on any of it.

AND YET IT DECIDES WHETHER THE THING WORKS

- a model that is accurate and too slow — does not work
- on a car at two metres per second, latency is distance
- L11 runs on a Jetson Nano — four gigabytes, and it is also powering the motors
- neither textbook covers this — and the absence is not an accident
- compression is practice attached to hardware, and it dates faster than theory

THREE TECHNIQUES

quantisation

fp32 → int8 — slides 18, 19

pruning

delete weights — slide 20

compilation

ONNX, TensorRT — slide 21

They compose. Quantise, then prune, then compile, and measure after each one — because the gains are not independent and the losses are not either.

Checked against the [Waveshare JetRacer Pro](#) page, 17 Aug 2026.

Quantisation — fp32 to int8

THE SAME ARITHMETIC AS AN ANALOGUE-TO-DIGITAL CONVERTER

- **a trained weight is a 32-bit float** — and almost none of those bits carry information
- **map the range onto 256 levels** — a scale and an offset
- four times smaller, and integer arithmetic is several times faster
- **the error model is familiar** — uniform rounding error, bounded by half a level
- **post-training conversion is nearly free** — quantisation-aware training costs a retrain

$$q = \text{round}(x/s) + z \quad \hat{x} = s(q - z)$$

A SIGNAL AT THREE BITS



WHAT IT BUYS

size	4 × smaller
memory traffic	4 × less — usually the binding one
arithmetic	integer, and faster

What It Costs Depends Entirely On the Metric

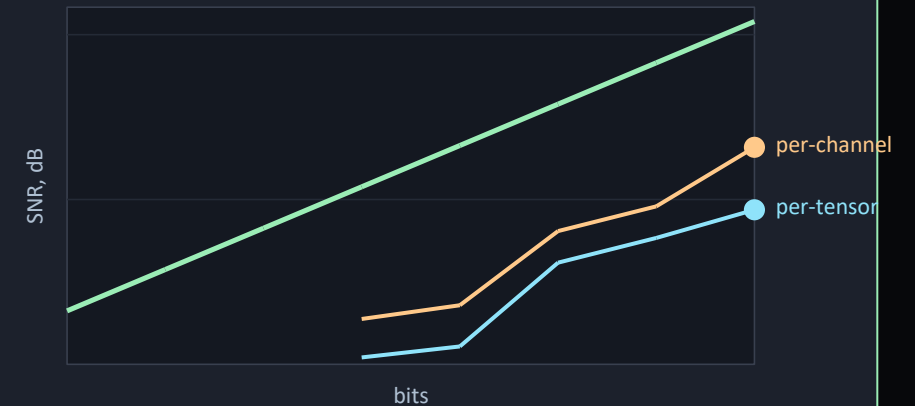
THE SAME CODE, THE SAME BIT WIDTH, TWO MODELS

A regression's error grows two hundredfold. A classifier loses nothing at all. If you quote the classifier's result and deploy the regression, you have misled yourself with a true number.

MEASURED, IN EX_06 NOTEBOOK 04

- **classification accuracy barely moves** — regression error moves a great deal
- **1.6e-06 to 3.5e-04** — over two hundred times worse
- **the reason is not subtle** — an argmax absorbs perturbation, a regression reports it
- **L11's steering network is a regression** — so the pessimistic column is the relevant one
- and per-channel scales buy a factor of eight, for 30 extra floats

SIGNAL-TO-NOISE AGAINST BIT WIDTH



$$s = \frac{x_{\max} - x_{\min}}{255} \quad \text{Var}(e) = \frac{s^2}{12}$$

Pruning — Deleting What Is Not Used

MOST OF THE WEIGHTS ARE DOING VERY LITTLE

- rank the weights by magnitude, zero the smallest, then fine-tune what is left
- **unstructured pruning gives high sparsity** — and no speedup without special hardware
- **structured pruning removes whole channels** — less sparsity, real speedup
- **for a Jetson Nano** — prune structurally, or do not bother
- and measure wall-clock latency, not the sparsity figure

STRUCTURED, NOT SCATTERED



one channel removed — the network is simply narrower

unstructured

high sparsity, no speedup

structured

lower sparsity, real speedup

what to measure

latency, not parameter count

The greyed node and its edges are gone. Because a whole channel went, the remaining matrices are smaller rather than sparser — which is the only version any ordinary GPU can exploit.

ONNX and TensorRT — Compiling for the Device

TRAINING FRAMEWORK OUT, DEVICE ENGINE IN

- **PyTorch is built for flexibility** — it rebuilds the graph every forward pass
- **ONNX is the interchange format** — a frozen graph, framework-independent
- **TensorRT compiles it** — fusing operations and choosing kernels for one GPU
- the speedup is often several-fold, for no accuracy change
- **and the engine is device-specific** — build it on the car, not on your laptop

THE TOOLCHAIN



torch2trt shortcuts the middle step and is what the JetRacer notebooks use.

WHAT THE COMPILER DOES

layer fusion	conv + bias + ReLU as one kernel
kernel selection	benchmarked on the actual GPU
precision	fp16 or int8 where it can

Thirty Frames Per Second, or Four

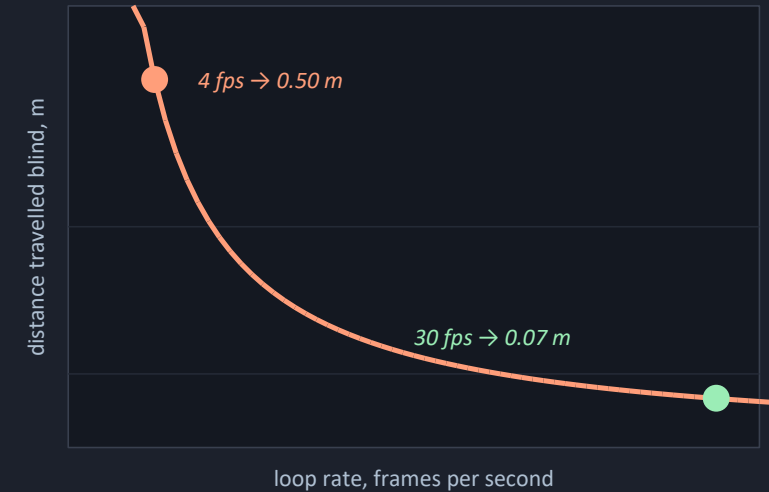
THE NUMBER THAT DECIDES WHETHER THE CAR WORKS

- the car moves at speed v — the loop takes time t
- so it travels v times t between decisions
- two metres per second at four frames — half a metre, on a three-metre track
- at thirty frames — seven centimetres, and the car behaves
- nothing about the model changed. Only how often it was asked.

$$d = v / f$$

Blind distance. Declare it, in metres, in the report.

BLIND DISTANCE AT 2 m/s



The curve is v over f and nothing else. Compression moves you along it; it does not change its shape.

Failure Modes After Training

FINE-TUNING MADE IT WORSE

Catastrophic forgetting — the learning rate was too large, or you unfroze too much. Divide the learning rate by ten and freeze everything but the head, then work back up.

THE AGENT LEARNED THE WRONG TASK

Reward hacking. It is doing exactly what you asked. Rewrite the reward as a specification, hand it to a colleague, and ask them how they would cheat it.

QUANTISED, AND THE ACCURACY HELD

Check which metric you looked at. An argmax hides what a regression cannot. Measure the thing you actually care about, in physical units — degrees of steering, not loss.

WHAT TO CHECK, IN ORDER OF COST

free	measure the loop rate, do not assume
free	compute the blind distance
free	state which metric you quoted
cheap	freeze more, lower the rate
cheap	compare int8 against fp32 directly
real work	collect recovery data

The last row is the imitation-learning failure from slide 13, and it is the one that costs a session. A model trained only on good driving has never seen the situation it will actually need to get out of.

Ex_06 — Notebooks 03, 04 and 05

TRANSFER, QUANTISE, AND THEN DEFEND A CHOICE

- **notebook 03** — transfer learning on a small image set
- **notebook 04** — float32 to int8, on a classifier and then on a regression
- **notebook 05** — the report, built from the files 01 to 04 write
- **the report asks for a latency budget** — measured, not assumed

THE NOTEBOOKS

00 – 02	L6.1's half, done last week
03	transfer, then fine-tune
04	float32 → int8, twice
05	the report
outputs	03 and 04 write .npz for 05

THE MARKED QUESTION FOR THIS HALF

Thirty labels could barely separate scratch, probe and fine-tune. Which model would you deploy, and what would have to be true for that to be right?

Key Takeaways

- 1 Training from scratch is the unusual case**

Start from pre-trained weights, replace the head, unfreeze a little at a tenth of the learning rate. Self-supervision is where those weights came from.
- 2 Reinforcement learning is a narrow tool**

It earns its cost when there is no demonstrator and a fast, faithful simulator. It has no stability guarantees, and you get exactly what you reward.
- 3 You already have the vocabulary**

State, action, policy, reward, return, value — all of them map onto control terms. The differences that matter are the plant model and the guarantees.
- 4 L11 imitates rather than reinforces**

No simulator, real crashes, a demonstrable task and one afternoon of data. The price is compounding error, so collect recovery data on purpose.
- 5 Deployment decides whether it works**

int8, structured pruning and TensorRT are the difference between 30 fps and 4 — and at 2 m/s that is 7 cm of blindness against 50.

Next — L7.1: everything L6.1 taught about optimisers, pointed at a physics residual.

Questions

TEN TO TEST WHETHER TODAY LANDED

- **when should you unfreeze more than the head** — and what decides it?
- what task does self-supervision invent, and why does that help you?
- **the three stages of post-training** — and which one is reinforcement learning?
- what is credit assignment, and why is a late aggregate signal hard?
- **imitation or reinforcement for the JetRacer** — and what do you give up?
- why does quantisation barely hurt classification and badly hurt regression?

AND FOUR MORE

seven

why is the Markov assumption doing real work?

eight

read γ as a time constant — set it for your problem

nine

unstructured or structured pruning, on a Jetson Nano?

ten

a model at four frames per second — how far does the car travel?

THE ONE WITH NO SETTLED ANSWER

Your model is accurate and too slow. Do you shrink it, compile it, or slow the car down?

BEFORE L7.1

Notebooks 03 to 05. Bring your measured latency.

References, and One Section With None

THE TWO BOOKS

Prince, Understanding Deep Learning — chapter 9.3 for transfer learning, fine-tuning and self-supervised pre-training, which is the whole of slides 3 to 5. Chapter 19 for reinforcement learning: 19.1 for the Markov decision process, 19.2 for return and value, 19.4 and 19.5 for policy gradient. Chapters 8 and 9 are the evaluation and regularisation material, which is L4.2's, not today's.

Goodfellow, Bengio & Courville — chapter 11, Practical Methodology, which is a decision procedure for what to do when your model underperforms. There is nothing like it in Prince and it is the better read for a working engineer. Read it before your project, not during it.

AND THE SEGMENT WITH NO BOOK AT ALL

Slides 17 to 22 are NVIDIA's TensorRT documentation and the PyTorch quantisation guide. Vendor documentation. It is authoritative for its own hardware, it is not peer reviewed, and it changes. Check the current version before you quote a number from today's slides.

Waveshare's JetRacer Pro product page for the hardware figures, checked 17 August 2026. The NVIDIA JetRacer and JetBot projects for the method — read them for the approach, not for the motor code.

READ IN THIS ORDER

before Ex_06 nb 03	L4.2 slides 17 – 19 again
before Ex_06 nb 04	the PyTorch quantisation guide
if slide 12 was fast	UDL ch. 19.4
before your project	GBC ch. 11 — all of it
before L7.1	L6.1 slides 20 – 23

NEXT

L7.1 — Physics-Informed Neural Networks

Part 1 ends here. Everything L6.1 taught you about optimisers is about to be pointed at a physics residual — the same Adam, the same L-BFGS, the same loss curve, and no data in the loss at all.

BEFORE THE NEXT SESSION

read	L6.1 slides 20 – 23, again
run	Ex_06 notebooks 03, 04 and 05
bring	your int8 comparison, both metrics
think about	what your residual would be

Six blocks of general deep learning are finished. From L7 every network you build has an equation in its loss, and the four bridge slides from last week are the only preparation you need.